

When LLM-Generated Code Perpetuates User Interface Accessibility Barriers, How Can We Break the Cycle?

Alexandra-Elena Guriță and Radu-Daniel Vatavu

Problem

- LLMs in web development workflows is transforming current practices in design and deployment of UIs for interactive computer systems
- However, 84% of websites generated using ChatGPT exhibited accessibility violations
 - text resizing (31.8%), contrast (26.6%), conveying semantic relationships (16.8%)
- LLMs trained on vast datasets turned out to be coding tools that propagate accessibility barriers instead of reducing them

Contributions

1. Tested the hypothesis that structured prompt engineering with specific WCAG success criteria can improve compliance across multiple accessibility dimensions
 - a. Comparing accessibility-agnostic with accessibility-oriented prompts
2. Examined LLMs' potential for self accessibility evaluation and remediation

Experiment

- Factorial design involving two independent variables: **LLM-Type** (Claude 3.5 Haiku and Chat GPT GPT-4-turbo) and **prompt-type** (accessibility-agnostic and accessibility-oriented prompts)
 - With each combination, they generated ten UIs and asked LLMs for a self-reflection on the accessibility of the UIs (doubled size of the dataset)

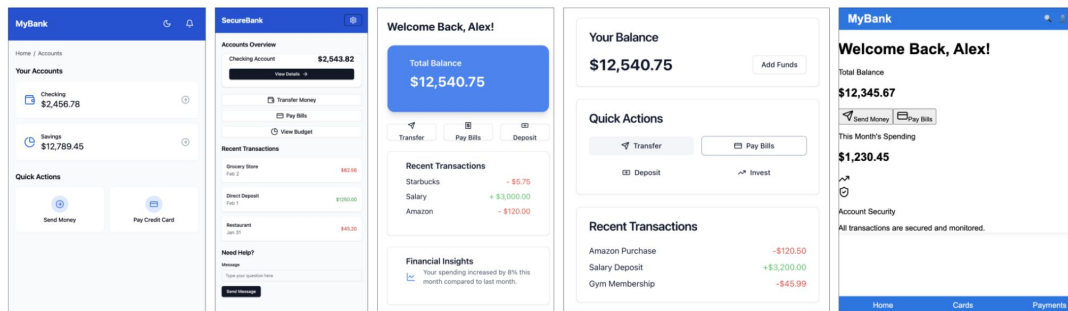


Figure 1: Examples of banking application user interfaces, from the dataset evaluated in this work, generated using ChatGPT and Claude code. Note common patterns, such as prominent balance displays, quick-access payment buttons, and recent transaction lists as well as the consistent use of blue color schemes and similar mobile-first navigation structures. These examples show how LLMs can successfully interpret banking application requirements but primarily replicate existing design patterns, thus perpetuating accessibility issues such as hard-to-read text, poor contrast ratios, and small touch targets.

Experiment (continued)

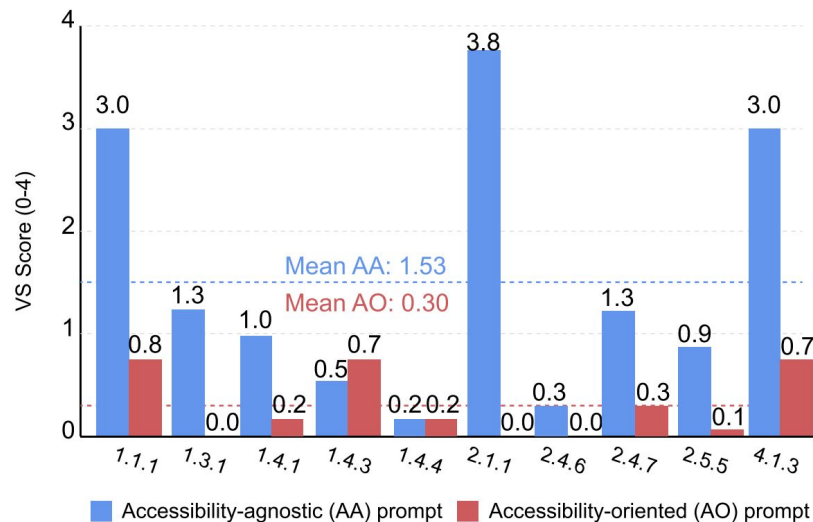
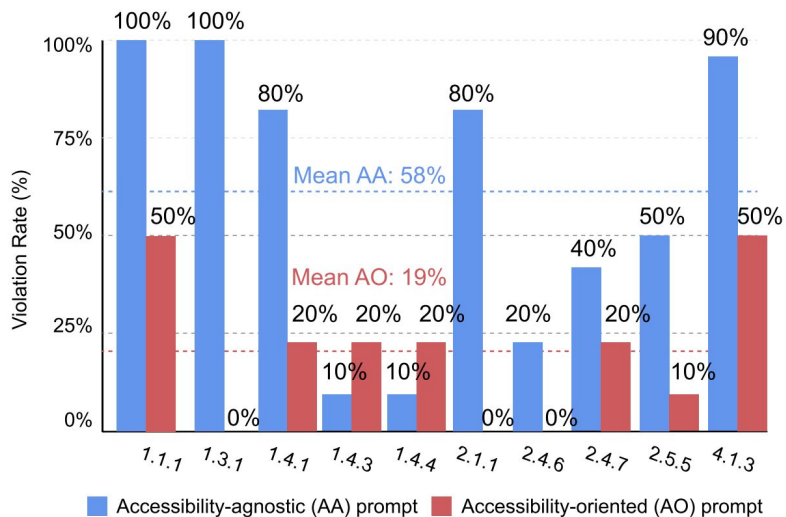
- Prompt engineering
 - accessibility-agnostic prompt (control): “Design the homepage of a banking app”
 - accessibility-oriented prompt (explicit): “Design an accessible banking app homepage with resizable text (no loss of functionality at 200% zoom), 4.5:1 minimum contrast ratio, clear semantic structure (proper headings, ARIA landmarks, labeled form controls), and 44x44px minimum touch targets per WCAG 2.1 AA”
- In total they generated 80 UIs where half were subjected to multi-faceted evaluation and half represent improvements of the first set

Evaluation

- Combined automated assessment (MAUVE++ with 50 WCAG criteria) and expert review (5-point severity scale)
- LLM self reflection
- UI's generated with the accessibility-oriented prompt generally performed better across important accessibility criteria (focus visibility, info & relationships, and parsing)
- Both prompt types succeeded in meeting the text contrast criteria (1.4.3), with high success counts across all conditions, suggesting good color contrast handling in the LLM-generated UIs

Evaluation (continued)

- Violation rates and violation severity
 - Accessibility-oriented prompt achieved substantially better compliance across most criteria



Evaluation (continued)

- Code comparison between prompt types
 - UI's generated with the accessible-oriented prompt showed better structural organization and robust implementation in terms of interactive elements
- LLM self-evaluation
 - LLMs prioritize criteria based on Level A and AA requirements, common accessibility barriers, and features with the highest user impact
 - “I start with these core WCAG criteria because they cover critical accessibility barriers that impact users the most.”
 - Identifies limitations: “Some WCAG success criteria don't apply to every interface”

Research Gap

- Automated evaluation was based on WCAG and the MAUVE++ tool which may not capture all aspects of accessibility implementation
- Evaluation was carried out on a small set of UIs generated by two LLMs
- Did not assess the long-term sustainability or robustness of the corresponding code to further modifications by developers

Future Scope

- Future work opportunities in LLM-based code generation for more accessible UIs
 - Explore methods for developing accessibility-focused training datasets that accurately represent not only accessible design patterns but also actual user needs
 - More research into how AI design tools can develop an understanding of accessibility principles
 - Explore novel approaches for teaching AI models about the underlying purposes of accessibility features rather than just their technical implementation
 - Understanding how prompt effectiveness vary across different UI types and AI models could enable more effective prompt engineering strategies

Future Scope (continued)

- Recommend developers using LLMs for UI generation to (1) explicitly include WCAG success criteria in prompts, (2) perform supplementary testing for keyboard navigation and focus management, and (3) leverage LLMs' self-reflection capabilities as an initial accessibility check while recognizing their limitations in assessing complex interactive requirements.
- Accessibility challenges have shifted from primarily visual presentation issues to more structural, semantic, and interactive aspects of the UIs
 - Progress is neither uniform nor complete